CI/CD Learning Roadmap — Foundational Understanding

Objective

The purpose of this roadmap is to build a strong foundational understanding of CI/CD and the overall software delivery lifecycle.

The focus is not only on learning tools or writing pipeline configurations, but on understanding:

- how software moves from development to production
- why automation is needed
- what problems CI/CD solves
- how modern deployment systems work internally

The roadmap is divided into modules to gradually build understanding from basic concepts to production-level workflows.

---

1. Software Delivery Fundamentals

This module focuses on understanding the complete journey of software delivery.

A developer writes code locally, collaborates with others using Git, creates pull requests, and eventually ships updates to users. As applications grow, manually handling deployments and validations becomes slower and error-prone.

This module introduces:

- Git workflow
- branches and pull requests
- development, staging, and production environments
- why deployment processes need structure and automation

The goal is to understand the operational flow of software before introducing CI/CD tools.

---

2. Build Systems & Artifacts

Applications are usually not deployed directly from raw source code.

Before deployment, code often goes through:

- transpilation
- bundling
- optimization
- packaging

This process generates deployable output known as an artifact.

Examples include:

- React production builds
- compiled backend applications
- Docker images

This module explains:

- what actually gets deployed
- why build systems are necessary
- how applications are prepared for production environments

Understanding artifacts is important because CI/CD pipelines usually work around building and distributing these outputs.

---

3. Continuous Integration (CI)

Continuous Integration focuses on automatically validating code changes whenever developers push updates.

Instead of manually checking everything, automated pipelines can:

- run tests
- perform linting
- validate types
- verify builds

This ensures that broken or unstable code is identified early before being merged into the main codebase.

The core idea of CI is:

- improving code reliability
- reducing integration issues
- maintaining development quality consistently

This module helps build understanding of automated validation workflows.

Example CI Workflow

Consider a MERN application with:

- React frontend
- Node/Express backend
- MongoDB database

A developer works on a feature branch such as:

feature/login-fix

After completing changes, the developer pushes code to GitHub and creates a pull request:

feature/login-fix → dev

As soon as the pull request is created, the CI pipeline automatically starts.

The pipeline may:

- install dependencies
- run tests
- perform linting
- verify production builds

If any validation fails:

- the CI pipeline fails
- the pull request may be blocked from merging into the `dev` branch

This helps prevent unstable code from reaching shared branches.

If all checks pass:

- merge becomes allowed

Later, similar validations may run again while merging:

dev → main

This creates multiple automated quality checkpoints before production deployment.

---

4. Continuous Deployment / Continuous Delivery (CD)

After code is validated, it needs to be released to staging or production environments.

Continuous Deployment and Continuous Delivery automate this release process.

This module explains:

- how applications are deployed automatically
- how updates reach servers or hosting platforms
- how deployment pipelines reduce manual effort and release risk

The focus is on understanding how software transitions from:

- validated code
  to
- live production systems

This is where automation begins directly impacting end users.

Example CD Workflow

Once CI validation passes successfully:

- deployment pipelines may automatically trigger

Example:

- frontend deploys to Vercel
- backend deploys to Render or AWS

The workflow becomes:

Developer pushes code

→ CI validates code

→ tests/build pass

→ CD deploys application

→ users receive updated version

Without CD:

- developers may manually upload files
- restart servers manually
- deploy updates inconsistently

CD helps standardize and automate this process.

---

5. CI/CD Tools and Workflow Engines

CI/CD automation is managed through workflow tools such as:

- GitHub Actions
- Jenkins
- GitLab CI
- CircleCI

These tools define:

- when automation runs
- what steps execute
- how deployments happen

This module introduces concepts like:

- workflows

- jobs
- runners
- triggers
- secrets management

The goal is not to memorize syntax immediately, but to understand how automation systems coordinate software delivery tasks.

Pipeline Configuration Basics

Most CI/CD workflows are defined using YAML configuration files.

Example:

```
name: CI Pipeline


on:

  push:

    branches:

      - main
```

YAML mainly defines:

- workflow structure
- triggers
- execution order

Actual logic is usually executed through:

- shell commands
- npm scripts
- Docker commands
- Node.js or Python scripts

Example:

```
- name: Run tests
```

run: npm test

In this case:

- YAML orchestrates the workflow
- npm executes the actual application logic

Reusable CI Actions

Modern CI systems also support reusable automation modules.

Example:

uses: actions/checkout@v4

This does not refer to project code.

It is a reusable GitHub Action provided by GitHub itself.

Its purpose is to:

- download the repository code into the CI runner machine

This is necessary because CI runners initially start as empty environments.

Reusable actions help simplify common automation tasks instead of rewriting everything manually.

---

6. Docker & Containers

Modern deployment systems heavily rely on containers.

Docker packages applications along with their dependencies into isolated, portable environments called containers.

This helps solve issues where applications behave differently across machines or servers.

This module introduces:

- Docker images
- containers
- Dockerfiles
- ports
- environment consistency

The focus is understanding why containers became foundational in modern backend and deployment workflows.

---

7. Deployment Platforms & Infrastructure

Once applications are built, they need infrastructure to run.

This module explains:

- where applications are hosted
- how servers and cloud platforms work
- how frontend, backend, databases, and networking connect together

Examples may include:

- Vercel
- Render
- Railway
- VPS hosting
- container hosting platforms

The objective is to understand how production applications are actually served to users.

---

8. Production Engineering Concepts

Deployment is not the final step of software delivery.

Production systems require:

- monitoring
- logging

- rollback strategies
- health checks
- secrets management
- observability

This module introduces the operational side of engineering after applications go live.

The goal is to understand:

- how production systems remain stable
- how failures are detected
- how deployments are recovered safely
- how reliability is maintained at scale

This forms the bridge between simple deployment knowledge and real-world production engineering practices.

Automated Testing Fundamentals

Automated testing is one of the most important parts of CI workflows.

Developers write tests manually inside the project repository.

Examples:

- backend API tests
- frontend component tests
- business logic tests
- integration tests

These tests are then executed automatically by the CI pipeline whenever code changes are pushed.

Example workflow:

Developer pushes code

→ CI starts

→ automated tests run

→ pass/fail result returned

If tests fail:

- merge may be blocked
- deployment may stop

The purpose is to catch issues early without relying entirely on manual testing.

Important Understanding About Testing

Tests are not limited to mathematical or calculation functions.

Real-world testing commonly validates:

- authentication flows
- APIs
- UI behavior
- integrations
- recommendation systems
- business rules
- user interactions

Testing fundamentally means:

- verifying expected behavior under different conditions

The goal is to increase confidence in software reliability while reducing production issues.

Overall Learning Outcome

By progressing through these modules, the objective is to build a clear mental model of:

- how modern software delivery works
- how CI/CD systems automate workflows
- how applications are validated, built, deployed, and maintained

The emphasis is on conceptual understanding first, followed by tooling and implementation later.

This approach helps develop practical engineering intuition instead of only learning configuration syntax or platform-specific workflows.